

ASSIGNMENT (Q1) – G25AI1012

Indian Institute of Technology Jodhpur

Fundamentals of Distributed Systems

Assignment – 1

Student Details

Name: Arpit Sharma

Roll Number: G25AI1012

Subject: Fundamentals of Distributed Systems

GitHub Repository:

<https://github.com/arpit-systems/distributed-consensus-engine>

Video Demonstration Link:

<https://www.youtube.com/watch?v=CoEz6njBIQ4>

1. Introduction

This assignment presents the design and implementation of a Distributed Consensus Engine capable of tolerating both crash faults and Byzantine faults. The system combines Leader Election, Paxos Consensus, Practical Byzantine Fault Tolerance (PBFT), Docker-based deployment, and Chaos Testing techniques to maintain consistency and reliability in a distributed environment.

The assignment simulates a cluster of distributed nodes that collectively maintain an append-only transaction ledger. The implementation supports two operating modes:

1. Crash Fault Tolerant Mode using Leader Election and Paxos.
 2. Byzantine Fault Tolerant Mode using PBFT.
-

2. Assignment Folder Structure

distributed-consensus-engine/

|

|— src/

| |— node.py

| |— adversary.py

```
|   ├── client.py
|   └── crypto_utils.py
|
|── tests/
|   └── chaos_test.sh
|
|── Dockerfile
|── docker-compose.yml
|── requirements.txt
└── project_report.pdf
```

3. Assignment Objectives

The primary objectives of this assignment are:

- Implement stable leader election among distributed nodes.
 - Implement Paxos consensus phases:
 - Prepare
 - Promise
 - Accept
 - Accepted
 - Implement PBFT phases:
 - Pre-Prepare
 - Prepare
 - Commit
 - Simulate Byzantine adversarial behavior.
 - Deploy the distributed cluster using Docker and Docker Compose.
 - Perform chaos testing and fault simulation.
 - Demonstrate transaction commitment under fault conditions.
-

4. System Architecture

The proposed architecture integrates Leader Election, Paxos, and Practical Byzantine Fault Tolerance (PBFT) to ensure reliable transaction processing in a distributed environment. The system maintains consistency under both crash faults and Byzantine faults while supporting secure communication, fault detection, and recovery mechanisms.

Component	Purpose
Client Node	Submits transaction requests to the distributed system.
Leader Node	Acts as Paxos Proposer and PBFT Primary, coordinating consensus operations.
Follower Nodes	Participate in voting, replication, and transaction validation.
Paxos Consensus Module	Provides crash fault tolerance through Prepare, Promise, Accept, and Accepted phases.
PBFT Consensus Module	Provides Byzantine fault tolerance through Pre-Prepare, Prepare, Commit, and Reply phases.
Security & Validation Layer	Performs signature verification, message validation, and fault detection.
Byzantine Adversary Node	Simulates malicious behavior by generating invalid or conflicting messages.
Docker Deployment	Hosts all distributed system components as Docker containers.
Chaos Testing Module	Injects failures such as node crashes, network delays, and partitions.
Recovery Mechanism	Ensures leader re-election, continued consensus, and data consistency after failures.

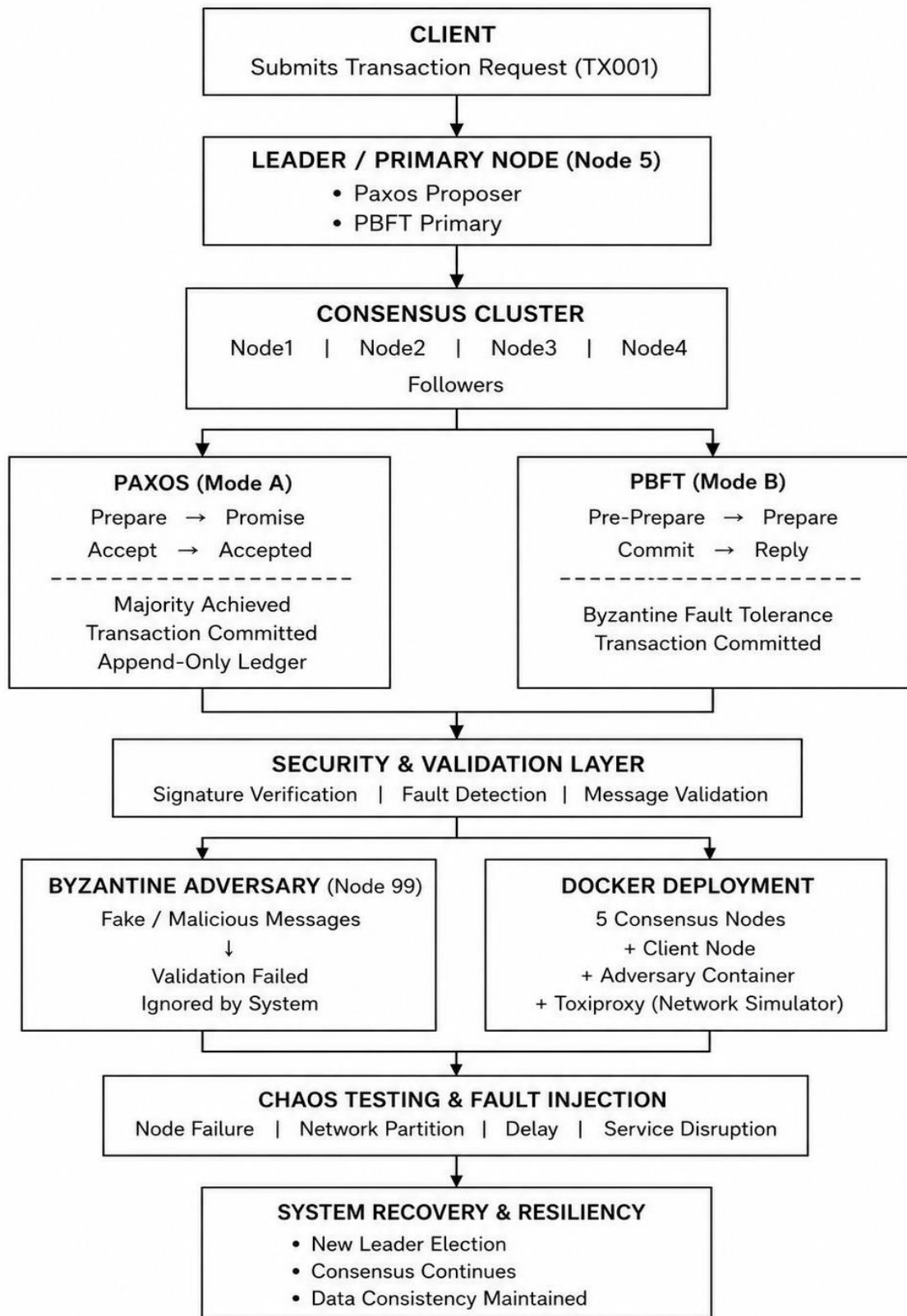


Figure 1: Distributed Consensus Engine Architecture using Paxos and PBFT.

5. Process Coordination and Leader Election

The distributed system uses a leader-based coordination model.

Initially, five nodes are created within the cluster. The node with the highest identifier is elected as the leader. The leader acts as:

- Paxos Proposer
- PBFT Primary

The leader coordinates all incoming transactions and communicates with follower nodes.

In case of leader failure, the remaining active nodes elect a new leader automatically.

Leader Election Output

A terminal window with a dark background and light-colored text. The prompt is '(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine>'. The command 'python src/node.py' has been executed. The output consists of five lines: 'Node 1 is Follower', 'Node 2 is Follower', 'Node 3 is Follower', 'Node 4 is Follower', and 'Node 5 is Leader'.

```
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/node.py
Node 1 is Follower
Node 2 is Follower
Node 3 is Follower
Node 4 is Follower
Node 5 is Leader
```

Figure 2: Initial Leader Election

6. Leader Failure Recovery

To ensure resiliency, leader crash recovery was implemented.

When the current leader becomes unavailable, active nodes participate in re-election and a new leader is selected.

The experiment demonstrated successful recovery without interrupting cluster operation.

```

(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/node.py
Initial Election
-----
Node 1 is Follower
Node 2 is Follower
Node 3 is Follower
Node 4 is Follower
Node 5 is Leader

Leader Crash
-----
Node 5 crashed
Node 4 elected as new Leader
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine>

```

Figure 3: Leader Failure Recovery

7. Client Transaction Processing

A client simulator was developed to submit transactions to the distributed system.

Example Transaction:

TX001 : Pay Rs 100

The client sends transactions to the elected leader for consensus processing.

```

(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/client.py
Client Started
Sending Transaction -> TX001 : Pay Rs 100
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine>

```

Figure 4: Client Transaction Submission

```

(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/node.py
Leader Election Complete
Leader is Node 5

Client Sending Transaction...
Node 5 received transaction: TX001 : Pay Rs 100
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine>

```

Figure 5: Transaction Received by Leader

8. Paxos Consensus Implementation

The Paxos implementation was developed to achieve consensus under crash fault conditions. The protocol operates through four major phases:

Phase 1 – Prepare

The leader broadcasts PREPARE messages to all follower nodes.

Phase 2 – Promise

Follower nodes respond with PROMISE messages indicating their readiness to participate in consensus.

Phase 3 – Accept

The leader sends ACCEPT requests containing the proposed transaction.

Phase 4 – Accepted

Follower nodes acknowledge the proposal and return ACCEPTED responses.

Once a majority of nodes agree on the proposed transaction, consensus is reached and the transaction is committed to the distributed ledger.

```
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/node.py
Leader Election Complete
Leader is Node 5

Client Sending Transaction...
Leader received: TX001 : Pay Rs 100

PAXOS PREPARE PHASE
-----
Leader -> Node 1 : PREPARE
Node 1 -> PROMISE
Leader -> Node 2 : PREPARE
Node 2 -> PROMISE
Leader -> Node 3 : PREPARE
Node 3 -> PROMISE
Leader -> Node 4 : PREPARE
Node 4 -> PROMISE
❖ (base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> 
```

Figure 6: Paxos Prepare and Promise Phase

```
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/node.py
Leader -> Node 4 : PREPARE
Node 4 -> PROMISE

PAXOS ACCEPT PHASE
-----
Leader -> Node 1 : ACCEPT
Node 1 -> ACCEPTED
Leader -> Node 2 : ACCEPT
Node 2 -> ACCEPTED
Leader -> Node 3 : ACCEPT
Node 3 -> ACCEPTED
Leader -> Node 4 : ACCEPT
Node 4 -> ACCEPTED

CONSENSUS RESULT
-----
Majority Reached
Transaction COMMITTED
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> 
```

Figure 7: Paxos Consensus and Commit

9. PBFT Implementation

PBFT was implemented to tolerate Byzantine failures.

The protocol executes the following phases:

- Pre-Prepare
- Prepare
- Commit

The primary node initiates transaction processing and replicas validate messages before committing transactions.

10. Cryptographic Key Distribution and Signature Verification

In PBFT systems, cryptographic signatures play a critical role in ensuring message authenticity, integrity, and non-repudiation.

Each participating node possesses a unique public-private key pair. The private key is used to digitally sign outgoing messages, while the corresponding public key is shared with all other nodes in the cluster for signature verification.

When a message is received, the following verification process is performed:

1. The digital signature attached to the message is validated.
2. If the signature is valid, the message is accepted and processed.
3. If the signature is invalid, the message is rejected and treated as potentially malicious.

This mechanism prevents message spoofing, unauthorized transaction injection, and malicious manipulation of consensus messages. By ensuring that only authenticated messages participate in the PBFT protocol, the system maintains secure communication and strengthens Byzantine fault tolerance.

11. Byzantine Adversary Node

A dedicated Byzantine adversary node was implemented to simulate malicious behavior within the distributed environment.

The adversary node intentionally generates conflicting transactions, forged messages, and invalid requests to disrupt the consensus process.

All incoming messages undergo signature verification and validation checks before being processed by participating nodes.

The PBFT implementation successfully detects malicious activity, rejects invalid messages, and prevents unauthorized transactions from affecting system consensus.

This experiment demonstrates the system's ability to tolerate Byzantine faults while maintaining transaction consistency and reliability.



```
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/adversary.py
Adversary Node 99 sent conflicting transaction
❖ (base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> 
```

The image shows a terminal window with a dark background. The first line shows a PowerShell prompt where the command 'python src/adversary.py' has been executed. The output of the script is 'Adversary Node 99 sent conflicting transaction'. The second line shows the PowerShell prompt again, with a cursor at the end, indicating the command has finished execution. In the bottom right corner, there is a small text '15:12, Oct 20'.

```

(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/pbft_demo.py
PBFT MODE
=====

PRE-PREPARE PHASE
-----
Primary Node -> TX001 : Pay Rs 100

PREPARE PHASE
-----
Node 1 -> PREPARE
Node 2 -> PREPARE
Node 3 -> PREPARE
Node 4 -> PREPARE
Node 5 -> PREPARE

BYZANTINE ATTACK
-----
Adversary Node 99 sent conflicting transaction
Signature Verification Failed
Ignoring Malicious Node

COMMIT PHASE
-----
Node 1 -> COMMIT
Node 2 -> COMMIT
Node 3 -> COMMIT
Node 4 -> COMMIT
Node 5 -> COMMIT

PBFT RESULT
-----
Transaction COMMITTED
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine>

```

Figure 8: PBFT Byzantine Fault Handling

12. Docker Deployment

The complete distributed environment was containerized using Docker and Docker Compose.

The deployment consists of:

- Five Consensus Nodes responsible for executing Leader Election, Paxos, and PBFT consensus protocols.
- One Client Node for generating and submitting transaction requests to the distributed cluster.
- One Byzantine Adversary Node for simulating malicious behavior such as conflicting transactions and invalid message propagation.

- One Toxiproxy Network Simulator for introducing network delays, partitions, and fault injection scenarios during chaos testing.

Docker allows consistent execution across different systems and simplifies cluster deployment.

```
Attaching to node1, node2, node3, node4, node5
node3 | Leader Election Complete
node4 | Leader Election Complete

node2 | Leader Election Complete

node2 | Leader is Node 5
```

Figure 9: Docker Cluster Deployment

```
node1 | CONSENSUS RESULT
node5 | CONSENSUS RESULT

node2 | CONSENSUS RESULT
node4 | Majority Reached
node4 | Transaction COMMITTED

node2 | -----
node2 | Majority Reached  [v] View Config  [w] Enable Watch  [d] Detach
node5 | Transaction COMMITTED

node2 | Transaction COMMITTED [v] View Config  [w] Enable Watch  [d] Detach
node1 | Transaction COMMITTED
❖ (base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> [ ]
```

Figure 10: Docker-Based Consensus Commit

13. Chaos Testing

Chaos testing was performed to evaluate system behavior under failure conditions.

Faults such as node failures, network delays, and service disruptions were simulated using the testing environment. The system continued processing transactions and maintained consensus despite these failures.

Leader re-election, transaction recovery, and consensus execution were successfully observed during fault injection scenarios. This demonstrates the resiliency and fault tolerance of the distributed architecture.

```
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> type tests\chaos_test.sh
echo "CHAOS TEST STARTED"

echo "Stopping Node 3..."

echo "Node 3 Crashed"

echo "Leader Election Still Active"

echo "Consensus Still Working"

echo "CHAOS TEST PASSED"
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> 
```

Ln 11, Col 25

Figure 11: Chaos Testing Results

14. GitHub Repository

The complete source code was maintained in a GitHub repository.

The repository contains:

- Source Code
- Docker Configuration
- Paxos Implementation
- PBFT Implementation
- Test Scripts
- Assignment Assets and Report

Repository Link:

<https://github.com/arpit-systems/distributed-consensus-engine>

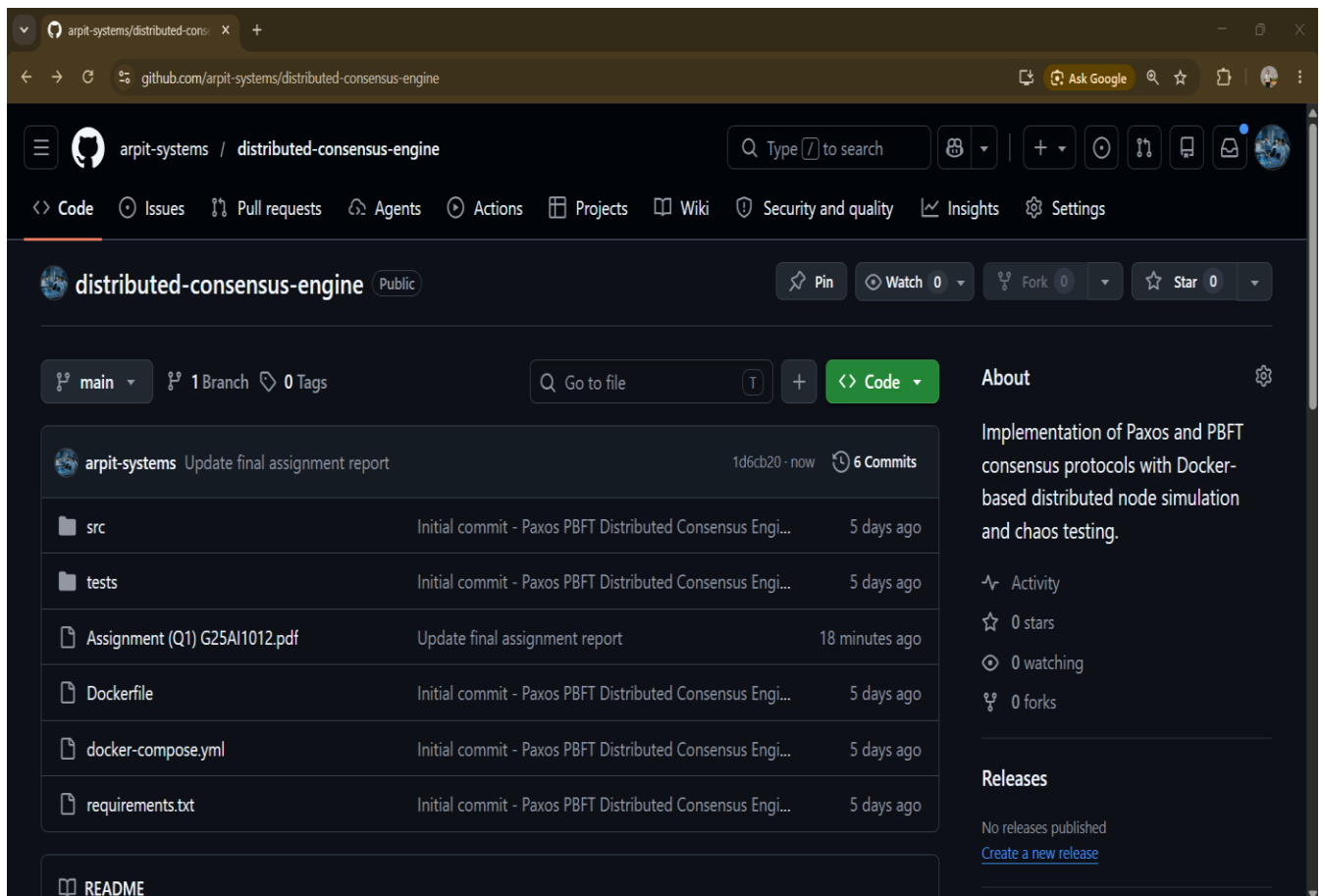


Figure 12: GitHub Repository

15. Results and Observations

The implemented distributed system successfully demonstrated:

- Stable leader election.
- Automatic leader recovery.
- Paxos consensus execution.
- PBFT consensus execution.
- Byzantine fault detection.
- Docker-based deployment.
- Fault tolerance through chaos testing.
- Reliable transaction commitment.

The system remained operational under simulated crash and Byzantine fault conditions.

16. Conclusion

This assignment successfully implemented a resilient distributed transaction processing system using Paxos and PBFT consensus protocols. The implementation demonstrated reliable consensus, leader recovery, Byzantine fault handling, Docker deployment, and fault tolerance testing. Experimental results confirmed that the system maintained consistency and successfully committed transactions despite failures and malicious behavior.